Audited by CRED SHIELDS

CredShields

Smart Contract Audit

April 15th, 2024 • CONFIDENTIAL

Description

This document details the process and result of the Vault Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Arcana between October 1st, 2024, and October 4th, 2024. A retest was performed on October 7th, 2024.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli(Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor)

Prepared for

Arcana

Table of Contents

Table of Contents **2**
1. Executive Summary ——————————- **3**
State of Security 4
2. The Methodology ——————————- **5**
2.1 Preparation Phase 5
2.1.1 Scope 5
2.1.2 Documentation 5
2.1.3 Audit Goals 6
2.2 Retesting Phase 6
2.3 Vulnerability classification and severity 6
2.4 CredShields staff 8
3. Findings Summary ——————————- **9**
3.1 Findings Overview 9
3.1.1 Vulnerability Summary 9
3.1.2 Findings Summary 11
4. Remediation Status ——————————- **14**
5. Bug Reports —————————— **16**
Bug ID #1 [Fixed] 16
Lack of withdrawal function leads to funds stuck in the vault contract 16
Bug ID #2 [Fixed] 17
Missing Handling of Fees on Transfer in ERC20 Token Transfers 17
Bug ID #3 [Fixed] 19
Network congestion may inflate gas price 19
Bug ID #4 [Fixed] 20
Gas Refund calculation may drain vaultBalance 20
Bug ID #5 [Fixed] 21
Storage Layout Conflict in Valut contract 21
Bug ID #6 [Fixed] 22
Missing Reentracy Guard modifier 22
Bug ID #7 [Fixed] 23
Use safeTransfer/safeTransferFrom instead of transfer/transferFrom 23
Bug ID #8 [Fixed] 25
Missing Events in Important Functions 25
Bug ID #9 [Fixed] 26
Floating and Outdated Pragma 26
Bug ID #10 [Fixed] 27

Use Call instead of Transfer 27
Bug ID #11 [Fixed] 29
Boolean Equality 29
Bug ID #12 [Fixed] 30
Splitting Require Statements 30
Bug ID #13 [Fixed] 31
Gas Optimization in Increments 31
Bug ID #14 [Fixed] 32
Cheaper Inequalities in if() 32
Bug ID #15 [Fixed] 33
Gas Optimization for State Variables 33
6. The Disclosure ——————————- 34

1. Executive Summary ----------

Arcana engaged CredShields to perform a smart contract audit from October 1st, 2024, to October 4th, 2024. During this timeframe, 15 vulnerabilities were identified. **A retest was performed on October 7th, 2024, and all the bugs have been addressed.**

During the audit, 1 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Arcana" and should be prioritized for remediation.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Vault Smart Contract	0	1	4	4	1	5	**15**
	0	**1**	**4**	**4**	**1**	**5**	**15**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Vault Smart Contract's scope during the testing window while abiding by the policies set forth by Arcana's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Arcana's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Arcana can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Arcana can future-proof its security posture and protect its assets.

2. The Methodology ----------

Arcana engaged CredShields to perform a Vault Smart Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from October 1st, 2024, to October 4th, 2024, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
• https://github.com/arcana-network/ca-sc/tree/3b949d83b022f81235f8d152b11bc0ed7d1d013e • https://github.com/arcana-network/ca-sc/tree/ffe3a81ba93d001b20680e5bcde8b3de34f039ee

2.1.2 Documentation

The following documentation was provided to help with the audit -

- https://blog.arcana.network/inside-the-demo-arcanas-chain-abstraction-explained/

2.1.3 Audit Goals

CredShields uses both in-house tools and manual methods for comprehensive smart contract security auditing. The majority of the audit is done by manually reviewing the contract source code, following SWC registry standards, and an extended industry standard self-developed checklist. The team places emphasis on understanding core concepts, preparing test cases, and evaluating business logic for potential vulnerabilities.

2.2 Retesting phase

Arcana is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by the asset and SWC classification. Each asset section will include a summary. The table in the executive summary contains the total number of identified security vulnerabilities per asset per risk indication.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 15 security vulnerabilities were identified in the asset.

VULNERABILITY TITLE	SEVERITY	SWC | Vulnerability Type
Lack of withdrawal function leads to funds stuck in the vault contract	High	Missing Withdrawal Logic
Missing Handling of Fees on Transfer in ERC20 Token Transfers	Medium	Calculation Inaccuracy
Network congestion may inflate gas price	Medium	Gas Price Manipulation
Gas Refund calculation may drain vaultBalance	Medium	Business Logic Issue
Storage Layout Conflict in Valut contract	Medium	Storage Layout Conflict
Missing Reentracy Guard modifier	Low	Reentrancy
Use safeTransfer/safeTransferFrom instead of transfer/transferFrom	Low	Missing Best Practices
Missing Events in Important Functions	Low	Missing Best Practices

Floating and Outdated Pragma	Low	Floating Pragma
Use Call instead of Transfer	Informational	Missing Best Practices
Boolean Equality	Gas	Gas Optimization
Splitting Require Statements	Gas	Gas Optimization
Gas Optimization in Increments	Gas	Gas Optimization
Cheaper Inequalities in if()	Gas	Gas Optimization
Gas Optimization for State Variables	Gas	Gas Optimization

Table: Findings in Smart Contracts

3.1.2 Findings Summary

SWC ID	SWC Checklist	Test Result	Notes
SWC-100	Function Default Visibility	Not Vulnerable	Not applicable after v0.5.X (Currently using solidity v >= 0.8.6)
SWC-101	Integer Overflow and Underflow	Not Vulnerable	The issue persists in versions before v0.8.X.
SWC-102	Outdated Compiler Version	Vulnerable	Bug ID #9
SWC-103	Floating Pragma	Vulnerable	Bug ID #9
SWC-104	Unchecked Call Return Value	Not Vulnerable	call() is not used
SWC-105	Unprotected Ether Withdrawal	Not Vulnerable	Appropriate function modifiers and require validations are used on sensitive functions that allow token or ether withdrawal.
SWC-106	Unprotected SELFDESTRUCT Instruction	Not Vulnerable	selfdestruct() is not used anywhere
SWC-107	Reentrancy	Vulnerable	Bug ID #6
SWC-108	State Variable Default Visibility	Not Vulnerable	Not Vulnerable
SWC-109	Uninitialized Storage Pointer	Not Vulnerable	Not vulnerable after compiler version, v0.5.0
SWC-110	Assert Violation	Not Vulnerable	Asserts are not in use.
SWC-111	Use of Deprecated Solidity Functions	Not Vulnerable	None of the deprecated functions like block.blockhash(), msg.gas, throw, sha3(), callcode(), suicide() are in use
SWC-112	Delegatecall to Untrusted Callee	Not Vulnerable	Not Vulnerable.

SWC-113	DoS with Failed Call	Not Vulnerable	No such function was found.
SWC-114	Transaction Order Dependence	Not Vulnerable	Not Vulnerable.
SWC-115	Authorization through tx.origin	Not Vulnerable	tx.origin is not used anywhere in the code
SWC-116	Block values as a proxy for time	Not Vulnerable	Block.timestamp is not used
SWC-117	Signature Malleability	Not Vulnerable	Not used anywhere
SWC-118	Incorrect Constructor Name	Not Vulnerable	All the constructors are created using the constructor keyword rather than functions.
SWC-119	Shadowing State Variables	Not Vulnerable	Not applicable as this won't work during compile time after version 0.6.0
SWC-120	Weak Sources of Randomness from Chain Attributes	Not Vulnerable	Random generators are not used.
SWC-121	Missing Protection against Signature Replay Attacks	Not Vulnerable	No such scenario was found
SWC-122	Lack of Proper Signature Verification	Not Vulnerable	Not used anywhere
SWC-123	Requirement Violation	Not Vulnerable	Not vulnerable
SWC-124	Write to Arbitrary Storage Location	Not Vulnerable	No such scenario was found
SWC-125	Incorrect Inheritance Order	Not Vulnerable	No such scenario was found
SWC-126	Insufficient Gas Griefing	Not Vulnerable	No such scenario was found
SWC-127	Arbitrary Jump with Function Type Variable	Not Vulnerable	Jump is not used.
SWC-128	DoS With Block Gas Limit	Not Vulnerable	Not Vulnerable.

SWC-129	Typographical Error	Not Vulnerable	No such scenario was found
SWC-130	Right-To-Left-Override control character (U+202E)	Not Vulnerable	No such scenario was found
SWC-131	Presence of unused variables	Not Vulnerable	No such scenario was found
SWC-132	Unexpected Ether balance	Not Vulnerable	No such scenario was found
SWC-133	Hash Collisions With Multiple Variable Length Arguments	Not Vulnerable	abi.encodePacked() or other functions are not used.
SWC-134	Message call with hardcoded gas amount	Not Vulnerable	Not used anywhere in the code
SWC-135	Code With No Effects	Not Vulnerable	No such scenario was found
SWC-136	Unencrypted Private Data On-Chain	Not Vulnerable	No such scenario was found

4. Remediation Status ----------

Arcana is actively partnering with CredShields from this engagement to validate the discovered vulnerabilities' remediations. **A retest was performed on October 7th, 2024, and all the issues have been addressed.**

Also, the table shows the remediation status of each finding.

VULNERABILITY TITLE	SEVERITY	REMEDIATION STATUS
Lack of withdrawal function leads to funds stuck in the vault contract	High	**Fixed** [Oct 7th, 2024]
Missing Handling of Fees on Transfer in ERC20 Token Transfers	Medium	**Fixed** [Oct 7th, 2024]
Network congestion may inflate gas price	Medium	**Fixed** [Oct 7th, 2024]
Gas Refund calculation may drain vaultBalance	Medium	**Fixed** [Oct 7th, 2024]
Storage Layout Conflict in Valut contract	Medium	**Fixed** [Oct 7th, 2024]
Missing Reentracy Guard modifier	Low	**Fixed** [Oct 7th, 2024]
Use safeTransfer/safeTransferFrom instead of transfer/transferFrom	Low	**Fixed** [Oct 7th, 2024]
Missing Events in Important Functions	Low	**Fixed** [Oct 7th, 2024]
Floating and Outdated Pragma	Low	**Fixed** [Oct 7th, 2024]
Use Call instead of Transfer	Informational	**Fixed** [Oct 7th, 2024]
Boolean Equality	Gas	**Fixed** [Oct 7th, 2024]

Splitting Require Statements	Gas	**Fixed** [Oct 7th, 2024]
Gas Optimization in Increments	Gas	**Fixed** [Oct 7th, 2024]
Cheaper Inequalities in if()	Gas	**Fixed** [Oct 7th, 2024]
Gas Optimization for State Variables	Gas	**Fixed** [Oct 7th, 2024]

Table: Summary of findings and status of remediation

5. Bug Reports ----------

Bug ID #1 [Fixed]

Lack of withdrawal function leads to funds stuck in the vault contract

Vulnerability Type
Missing Withdrawal Logic

Severity
High

Description
The Vault contract lacks a withdrawal function for native tokens, despite accepting them via the receive() function. These native tokens are intended for gas refunds to users, but without a withdrawal mechanism, the protocol cannot recover unused or excess funds from the contract. Over time, native tokens could accumulate beyond the amount required for refunds, becoming effectively locked within the contract.

Affected Code
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol

Impacts
While the funds are intended for gas refunds, if they exceed the refund requirements, there is no way for the protocol to reclaim the surplus or reallocate it for other necessary expenses.

Remediation
To remediate this, the contract should implement a withdrawal function that allows authorized parties to withdraw native tokens from the vault.

Retest
This vulnerability has been fixed by adding a withdraw() function.

Bug ID #2 [Fixed]

Missing Handling of Fees on Transfer in ERC20 Token Transfers

Vulnerability Type
Calculation Inaccuracy

Severity
Medium

Description
The deposit() function in the Vault contract does not account for ERC-20 tokens that implement fee-on-transfer mechanisms. Some weird ERC20 tokens deduct a fee during transfers, causing the actual amount received by the contract to be less than the amount specified by the user in the request. The Vault contract records this inflated value, believing that the full value is transferred. However, due to the fee charged during the transfer, the actual tokens received will be less than expected.

Scenario
1. Bob deposits 100 tokens into the Vault contract, but the tokens charge a transfer fee, so the Vault receives less than 100 tokens.
2. Despite receiving fewer tokens, the Vault still records Bob's deposit as 100 tokens.
3. A solver picks the transaction and calls the fill() function, transferring the full 100 tokens to the destination.
4. Since the destination token does not charge a transfer fee, the solver sends the full 100 tokens, effectively sending more than what the Vault received.

Affected Code
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L118-L122
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L175-L179

Impacts
This may result in users receiving less than their expected deposit/fill amounts.

Remediation
It is recommended that additional checks be implemented when receiving tokens to ensure that the actual tokens transferred to the Vault match the expected amount. This can be achieved by

checking the contract's balance before and after the transfer and adjusting the recorded value accordingly to account for any discrepancies caused by fee-on-transfer tokens.

Retest

This has been fixed by implementing validations on the actual amount of tokens transferred.

Bug ID #3 [Fixed]

Network congestion may inflate gas price

Vulnerability Type
Gas Price Manipulation

Severity
Medium

Description
In the deposit() function, the gas price used for refund calculations is determined by tx.gasprice, which reflects the gas price at the time of the transaction. During periods of network congestion, gas prices can spike significantly. This creates a vulnerability where users can trigger deposits during these high-gas periods, knowing that the protocol will refund the elevated gas costs. Since the user has no disincentive to transact at higher gas prices (due to the refund), this creates a situation where the protocol, through its vault balance, will bear the burden of inflated gas costs. As a result, the vault's funds can be drained more rapidly than intended, depleting resources allocated for legitimate operations or future transactions.

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L129

Impacts
The protocol may face significantly higher operational costs in gas refunds during periods of high network activity. Over time, this could reduce the protocol's available funds

Remediation
It is recommended to implement a cap on the gas price that is used for refund calculations. By setting a maximum gas price, the protocol can limit its exposure to sharp increases in gas prices during congestion.

Retest
This issue has been fixed by adding a maxGasPrice validation.

Bug ID #4 [Fixed]

Gas Refund calculation may drain vaultBalance

Vulnerability Type
Business Logic Issue

Severity
Medium

Description
The Vault contract's deposit function contains a flaw in its gas refund calculation that could result in the complete depletion of the vault balance. Specifically, if the calculated refund amount is close to or equal to the current vaultBalance, the contract will set vaultBalance to zero, even when the difference between the refund and the available balance is minimal.

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L130-L135

Impacts
Users will not get refunds for gas fees if the vaultBalance is low.

Remediation
It is recommended to update the refund logic to correctly handle cases where the refund amount is close to the vaultBalance. Rather than zeroing out the vault when the refund exceeds the balance, the contract should refund only up to the remaining balance, preserving the vault's state for future transactions.

Retest
This issue has been fixed by updating the refund logic.

Bug ID #5 [Fixed]

Storage Layout Conflict in Valut contract

Vulnerability Type
Storage Layout Conflict

Severity
Medium

Description
The Vault contract is upgradeable but lacks a storage gap. Without a storage gap, future versions of the contract may introduce new variables that overwrite storage slots in unexpected ways, potentially corrupting the contract's state. This issue arises from Solidity's use of sequential storage slots, where new variables can be written into slots reserved for other purposes in the original version of the contract.

Affected Code
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol

Impacts
Without a storage gap, new storage variables introduced in the Vault contract can overwrite the beginning of the storage layout, causing unexpected behavior and potentially severe vulnerabilities.

Remediation
Introduce a storage gap in the Vault contract to reserve space for future storage variables without affecting the contract's storage layout. Or you can use namespace variables.

Retest
This issue has been fixed by adding a storage gap in the contract.

Bug ID #6 [Fixed]

Missing Reentracy Guard modifier

Vulnerability Type
Reentracy

Severity
Low

Description
The deposit(), fill(), and settle() functions in the Vault contract lack the ReentrancyGuard modifier, which opens the contract to potential reentrancy attacks. Reentrancy occurs when an external contract makes a recursive call back into the vulnerable function before the original execution is completed. In this case, malicious actors could exploit this vulnerability by repeatedly calling the deposit, fill, or settle functions before their prior execution is finished, allowing them to manipulate the state of the contract in unexpected ways

Affected Code
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L86
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L138
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L208

Impacts
If an attacker can create a recursive call and re-enter either of these functions before they finish executing, they can potentially manipulate the state of the contract and exploit it to their advantage.

Remediation
To remediate this vulnerability, the contract should include a ReentrancyGuard modifier from OpenZeppelin or a similar mechanism on all functions that handle token transfers, balance updates, or state changes, such as deposit(), fill(), and settle().

Retest
This issue has been fixed by adding a nonReentrant modifier.

Bug ID #7 [Fixed]

Use safeTransfer/safeTransferFrom instead of transfer/transferFrom

Vulnerability Type
Missing Best Practices

Severity
Low

Description
The transfer() and transferFrom() method is used instead of safeTransfer() and safeTransferFrom(), presumably to save gas however OpenZeppelin's documentation discourages the use of transferFrom(), use safeTransferFrom() whenever possible because safeTransferFrom auto-handles boolean return values whenever there's an error.

Affected Code
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L118
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L175
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L190
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L244

Impacts
Using safeTransferFrom has the following benefits -
- It checks the boolean return values of ERC20 operations and reverts the transaction if they fail,
- at the same time allowing you to support some non-standard ERC20 tokens that don't have boolean return values.
- It additionally provides helpers to increase or decrease an allowance, to mitigate an attack possible with vanilla approve.

Remediation
Consider using safeTransfer() and safeTransferFrom() instead of transfer() and transferFrom().

Retest

This issue has been fixed as recommended in Remediation.

Bug ID #8 [Fixed]

Missing Events in Important Functions

Vulnerability Type
Missing Best Practices

Severity
Low

Description
Events are inheritable members of contracts. When you call them, they cause the arguments to be stored in the transaction's log—a special data structure in the blockchain. These logs are associated with the address of the contract which can then be used by developers and auditors to keep track of the transactions.

The contract was found to be missing these events on certain critical functions which would make it difficult or impossible to track these transactions off-chain.

Affected Code
The following functions were affected -

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L202-L206
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L251-L253

Impacts
Events are used to track the transactions off-chain and missing these events on critical functions makes it difficult to audit these logs if they're needed at a later stage.

Remediation
Consider emitting events for important functions to keep track of them.

Retest
This issue has been fixed by emitting events on the important functions.

Bug ID #9 [Fixed]

Floating and Outdated Pragma

Vulnerability Type
Floating Pragma (SWC-103)

Severity
Low

Description
Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.
The contract allowed floating or unlocked pragma to be used, i.e., ^0.8.13. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L2

Impacts
If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.
Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.
The likelihood of exploitation is low.

Remediation
Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.25 pragma version
Reference: https://swcregistry.io/docs/SWC-103

Retest
This issue has been fixed by updating the pragma version to the latest.

Bug ID #10 [Fixed]

Use Call instead of Transfer

Vulnerability Type
Missing Best Practices

Severity
Informational

Description:
Using Solidity's transfer function has some notable shortcomings when the withdrawer is a smart contract, which can render ETH deposits impossible to withdraw. Specifically, the withdrawal will inevitably fail when:

- The withdrawer smart contract does not implement a payable fallback function.
- The withdrawer smart contract implements a payable fallback function which uses more than 2300 gas units.
- The withdrawer smart contract implements a payable fallback function which needs less than 2300 gas units but is called through a proxy that raises the call's gas usage above 2300.

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L132
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L171
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L241

Impacts
The transfer function has some restrictions when it comes to sending ETH to contracts in terms of gas which could lead to transfer failure in some cases.

Remediation
It is recommended to transfer ETH using the call() function, handle the return value using require statement, and use the nonreentrant modifier wherever necessary to prevent reentrancy.
Ref: https://solidity-by-example.org/sending-ether/

Retest

This issue has been fixed as mentioned in the remediation.

Bug ID #11 [Fixed]

Boolean Equality

Vulnerability Type
Gas Optimization

Severity
Gas

Description
The contract was found to be equating variables with a boolean constant inside a "require()" statement which is not recommended and is unnecessary. Boolean constants can be used directly in conditionals.

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L104-L107
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L154-L157
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L234-L234

Impacts
Equating the values to boolean constants in conditions costs gas and can be used directly.

Remediation
It is recommended to use boolean constants directly. It is not required to equate them to true or false.

Retest
This issue has been fixed as recommended in remediation.

Bug ID #12 [Fixed]

Splitting Require Statements

Vulnerability Type
Gas Optimization

Severity
Gas

Description
Require statements when combined using operators in a single statement usually lead to a larger deployment gas cost but with each runtime calls, the whole thing ends up being cheaper by some gas units.

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L229-L233

Impacts
The multiple conditions in one require/revert statement combine require statements in a single line, increasing deployment costs and hindering code readability.

Remediation
It is recommended to separate the require statements with one statement/validation per line.

Retest
This issue has been fixed as recommended in the remediation.

Bug ID #13 [Fixed]

Gas Optimization in Increments

Vulnerability Type
Gas optimization

Severity
Gas

Description
The contract uses two for loops, which use post increments for the variable "i".
The contract can save some gas by changing this to ++i.
++i costs less gas compared to i++ or i += 1 for unsigned integers. In i++, the compiler has to create a temporary variable to store the initial value. This is not the case with ++i in which the value is directly incremented and returned, thus, making it a cheaper alternative.

Vulnerable Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L164
- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L239

Impacts
Using i++ instead of ++i costs the contract deployment around 600 more gas units.

Remediation
It is recommended to switch to ++i and change the code accordingly so the function logic remains the same and meanwhile saves some gas.

Retest
This issue has been fixed as recommended in remediation.

Bug ID #14 [Fixed]

Cheaper Inequalities in if()

Vulnerability Type
Gas Optimization

Severity
Gas

Description
The contract was found to be doing comparisons using inequalities inside the "if" statement. When inside the "if" statements, non-strict inequalities (>=, <=) are usually cheaper than the strict equalities (>, <).

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L130

Impacts
Using strict inequalities inside "if" statements costs more gas.

Remediation
It is recommended to go through the code logic, and, if possible, modify the strict inequalities with the non-strict ones to save gas as long as the logic of the code is not affected.

Retest:
This issue has been fixed as recommended in the remediation.

Bug ID #15 [Fixed]

Gas Optimization for State Variables

Vulnerability Type
Gas Optimization

Severity
Gas

Description
Plus equals (+=) costs more gas than the addition operator. The same thing happens with minus equals (-=). Therefore, `x +=y` costs more gas than `x = x + y`.

Affected Code

- https://github.com/arcana-network/ca-sc/blob/ffe3a81ba93d001b20680e5bcde8b3de34f039ee/contracts/Vault.sol#L252

Impacts
Writing the arithmetic operations in `x = x + y` format will save some gas.

Remediation
It is suggested to use the format `x = x + y` in all the instances mentioned above.

Retest
This issue has been fixed as recommended in the remediation.

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

YOUR SECURE FUTURE STARTS HERE

CRED SHIELDS

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

Audited by CRED SHIELDS

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com